FAST-NU

Department of Software Engineering

SOFTWARE RE-ENGINEERING

Project Report

Blog Platform

System Exploration, Reverse Engineering, Design Recovery, Weakness Analysis And Re-Engineering Proposal

GROUP MEMBERS	
Huzaifa Kashif	22K-5158
Muhammad Uzair	22K-5176
Awais Jamali	22K-5179

Course: Software Re-Engineering | FAST-NU

Note: OpenAI API key is currently not configured. AI-powered features are non-functional; the platform operates in standard mode only.

Table of Contents

Table of Contents 2
Step 1 — System Exploration 4
1.1 System Overview 4
1.2 Main Functionalities 4
1.3 Target Users 5
Step 2 — Reverse Engineering 6
2.1 System Architecture 6
2.2 Module Breakdown 6
Step 3 — Design Recovery (UML Diagrams) 8
3.1 Use Case Diagram 8
3.2 Domain Model 8
3.3 Class Diagram 8
3.4 DFD Level 0 9
3.5 DFD Level 1 9
Step 4 — System Weakness Analysis 11
4.1 Summary Table 11
4.2 Detailed Findings 11
W1 — No Input Sanitization on Post Content (Stored XSS) 12
W2 — AI Layer Fails Silently with No UI Status Indication 12
W3 — Missing manage.py and requirements.txt in the Repository 12
W4 — SQLite Used as the Database 12
W5 — Comment Deletion by Post Authors Has No Audit Trail 12
W6 — Hardcoded Fallback SECRET_KEY and Permanently Enabled DEBUG 12
W7 — No Pagination on Notifications; Bookmarks Hard-Limited to 10 13
W8 — No Validation on Image Uploads 13
W9 — Monolithic views.py (27 Functions, 600+ Lines) 13
W10 — No Test Suite 13
W11 — "No Comments Yet" Placeholder Not Removed on First AJAX Comment 13
Step 5 — Re-Engineering Proposal 14
5.1 Summary Table 14
5.2 Detailed Proposals 14
P1 — Sanitize Post Content with bleach to Prevent XSS 14
P2 — Add AI Feature Availability Flag and UI Status Banner 15
P3 — Commit manage.py, requirements.txt, and .env.example 15
P4 — Migrate from SQLite to PostgreSQL 15
P5 — Notify Comment Author When Their Comment Is Removed 16
P6 — Enforce Secure Settings via Startup Environment Validation 16
P7 — Paginate Notifications and Remove the Bookmark Hard-Limit 16

P8 — Validate Uploaded Images for Type, Size, and Integrity 16
P9 — Refactor views.py into a Feature-Based views/ Package 17
P10 — Write a Core Test Suite 17
P11 — Remove Placeholder Before Inserting First AJAX Comment 17
Repository Link For The Modified Blog Platform 19

Step 1 — System Exploration

1.1 System Overview

Blog Platform is a full-stack web application built with Django (Python 3.13), backed by SQLite, and served through Django's built-in development server. It enables users to create, publish, and manage blog posts, with support for Markdown authoring, social interactions, and content discovery via keyword and semantic search.

The project follows Django's MVT (Model-View-Template) architecture and is organized into two apps:

- blog — all content models, views, forms, signals, AI utilities, admin config, and URL patterns.
- blog_platform — project-level configuration: settings.py, root URLs, wsgi.py, and asgi.py.

An AI layer powered by OpenAI's GPT-4o-mini and text-embedding-3-small is present in the codebase to automate tag generation, category suggestion, SEO metadata, post summarization, and semantic search. Because the API key is currently unavailable, all AI features are non-operational and the platform runs in standard mode.

1.2 Main Functionalities

The platform supports three roles. The table below summarizes their permissions.

Feature	Guest	Registered User	Admin / Superuser
Browse and read posts	Yes	Yes	Yes
Keyword search and filter	Yes	Yes	Yes
View author profiles and About page	Yes	Yes	Yes
Register / login / reset password	Yes	—	—
Verify email	—	Yes	Yes
Create, edit, delete own posts	—	Yes	Yes
Save as draft	—	Yes	Yes
Like, comment, bookmark	—	Yes	Yes
Delete own / others' comments (own posts)	—	Yes	Yes
Receive notifications	—	Yes	Yes
Analytics dashboard	—	Yes	Yes
Edit profile	—	Yes	Yes
Semantic AI search *	—	Yes*	Yes*
Manage all users / posts / comments	—	—	Yes
AI-generate tags, SEO, summary *	—	—	Yes*
Manage categories and tags	—	—	Yes

* Features marked with an asterisk require the OpenAI API key, which is currently not configured.

1.3 Target Users

General Readers (Guests): Browse, read, search, and filter published content without an account. They interact in a read-only capacity.

Content Creators (Registered Users): Writers who publish posts, manage their content library, engage socially, and monitor performance via the analytics dashboard.

Platform Administrators (Superusers): Technical moderators who manage the entire platform via the Django admin panel (/admin), moderate any content, manage taxonomy, and access AI tools when the API key is available.

Step 2 — Reverse Engineering

2.1 System Architecture

The platform follows Django's MVT architecture — Django's interpretation of MVC. The Model defines data structures and handles all database interaction via the ORM; the View contains business logic and processes HTTP requests/responses; the Template is the HTML presentation layer returned to the browser.

Request-response cycle: a browser submits a request → the root URL dispatcher (blog_platform/urls.py) forwards it to the app-level router (blog/urls.py) → the matching view in blog/views.py executes business logic, queries models, and renders the appropriate template.

Key infrastructure notes:

- Database: SQLite (db.sqlite3) — managed entirely via the Django ORM. No raw SQL is written.
- Static assets: CSS and JavaScript served from static/. User-uploaded media stored under media/ (avatars/, post_covers/, post_gallery/).
- AI layer: Isolated in blog/ai_utils.py. Provides five functions (generate_embedding, cosine_similarity, generate_summary, generate_tags_and_category, generate_seo_metadata), all of which fail gracefully when no API key is configured.

2.2 Module Breakdown

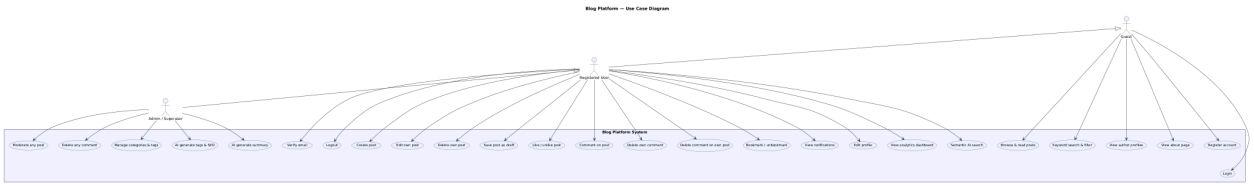
File / Module	Responsibility
manage.py	Django CLI entry point. Runs the development server, applies migrations, and creates superusers.
blog_platform/settings.py	Central configuration. Defines installed apps, database, static/media paths, email backend, and loads the OpenAI key from .env. SECRET_KEY falls back to a hardcoded insecure value if .env is absent.
blog_platform/urls.py	Root URL dispatcher. Includes blog.urls and maps /admin to the Django admin panel.
blog_platform/wsgi.py	WSGI entry point for production servers such as Gunicorn or uWSGI.
blog/models.py	Defines all ten database entities: Category, Tag, Post, PostImage, PostLike, Bookmark, Comment, Notification, UserProfile, and the built-in User.
blog/views.py	Contains all 27 view functions: authentication, post CRUD, social interactions, profile management, analytics, search, email verification, notifications, and two AI AJAX endpoints.
blog/urls.py	Maps all 27 URL patterns to their corresponding view functions.
blog/admin.py	Registers all models with the Django admin panel. PostAdmin adds three AI action buttons that trigger AJAX calls to the OpenAI API.
blog/ai_utils.py	Standalone OpenAI integration module. Provides generate_embedding(), generate_summary(), generate_tags_and_category(), generate_seo_metadata(), and cosine_similarity() using NumPy.

blog/signals.py	Listens for the post_save signal on User and auto-creates a UserProfile on every new registration.
blog/context_processors.py	Injects the unread notification count into every template's context so the navbar badge is always accurate.
blog/forms.py	Defines five Django forms: UserRegistrationForm, UserLoginForm, ProfileForm, CommentForm, and UserInfoForm.
blog/sitemaps.py	Generates an XML sitemap of all published posts for search engine indexing.
blog/migrations/	Five migration files (0001–0005) that define the complete database schema history.
templates/	All HTML templates: base.html, auth/, blog/ (with partials/), profile/, and pages/.
static/	Frontend CSS, JavaScript, and custom admin assets including the AI button scripts.
media/	Runtime directory for user-uploaded avatars, post covers, and gallery images.

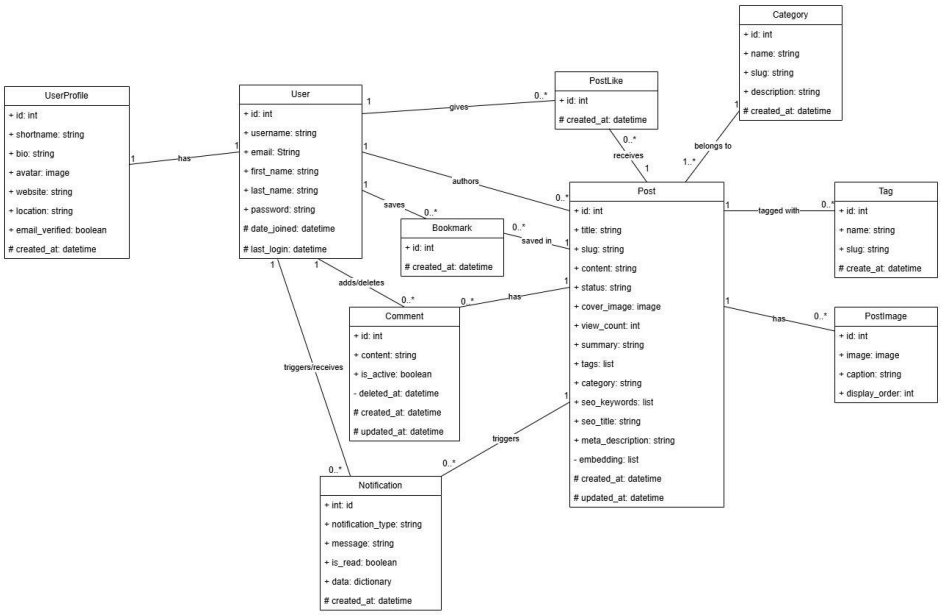
Step 3 — Design Recovery (UML Diagrams)

This step recovers the original design of the Blog Platform through four UML diagrams. The diagrams are presented below with explanatory notes; full-size renders were produced and presented separately during the project demonstration.

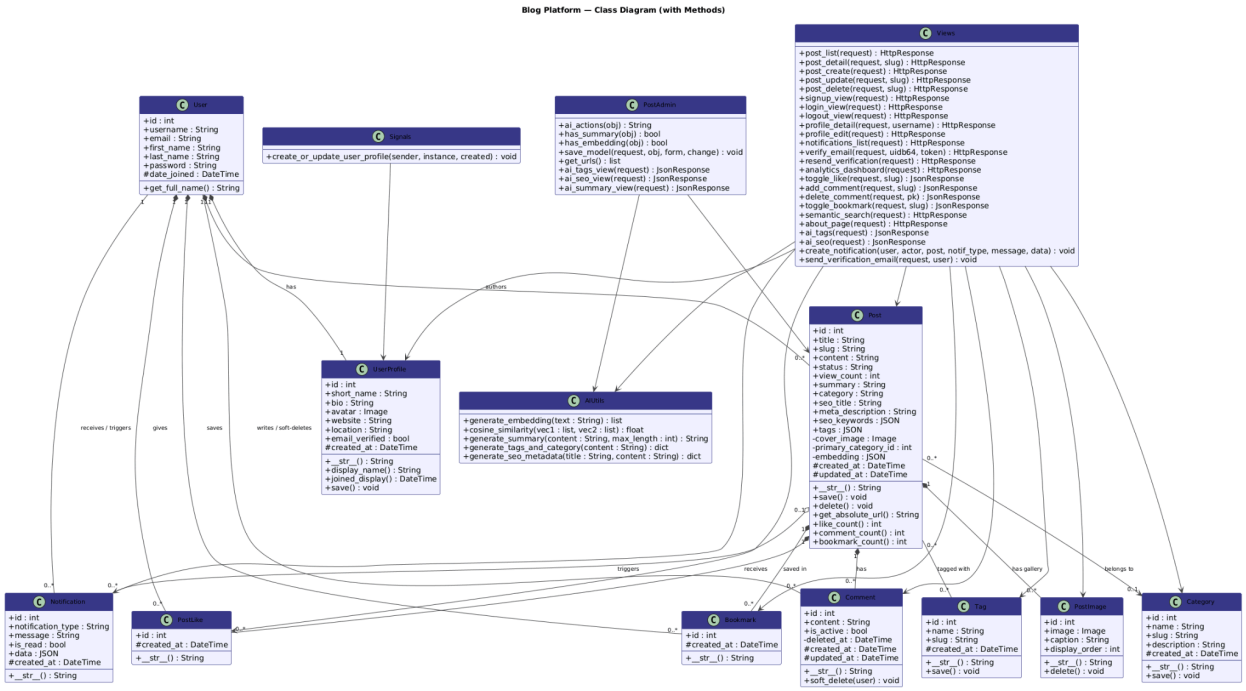
3.1 Use Case Diagram



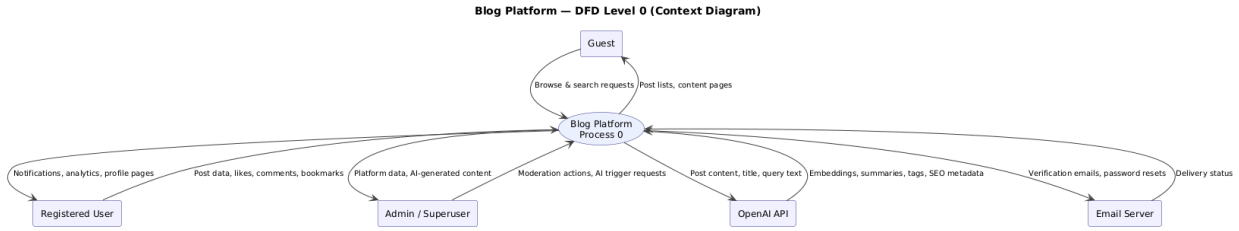
3.2 Domain Model



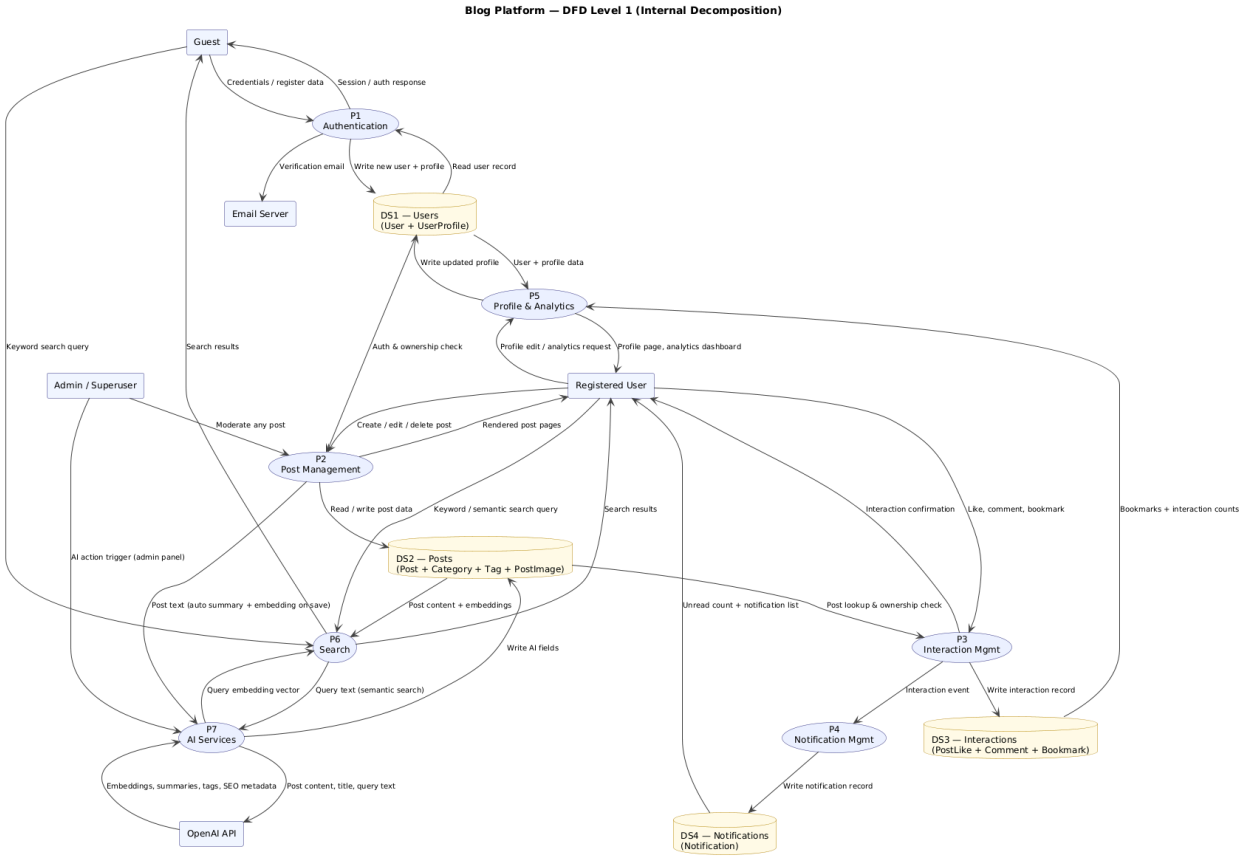
3.3 Class Diagram



3.4 DFD Level 0



3.5 DFD Level 1


Blog Platform — DFD Level 1 (Internal Decomposition)
Guest
Credentials / register data
Session / auth response
P1
Authentication
Verification email
Write new user + profile
Read user record
Email Server
DS1 — Users
(User + UserProfile)
Write updated profile
User + profile data
P5
Profile & Analytics
Keyword search query
Search results
Auth & ownership check
Profile edit / analytics request
Profile page, analytics dashboard
Admin / Superuser
Registered User
Moderate any post
Create / edit / delete post
Rendered post pages
P2
Post Management
Read / write post data
Keyword / semantic search query
Search results
Interaction confirmation
Like, comment, bookmark
Bookmarks + interaction counts
AI action trigger (admin panel)
DS2 — Posts
(Post + Category + Tag + PostImage)
Post text (auto summary + embedding on save)
Post content + embeddings
Unread count + notification list
Post lookup & ownership check
P6
Search
Write AI fields
P3
Interaction Mgmt
Query embedding vector
Query text (semantic search)
Interaction event
Write interaction record
P7
AI Services
P4
Notification Mgmt
DS3 — Interactions
(PostLike + Comment + Bookmark)
Embeddings, summaries, tags, SEO metadata
Post content, title, query text
Write notification record
OpenAI API
DS4 — Notifications
(Notification)

Step 4 — System Weakness Analysis

This section documents ten concrete weaknesses identified through direct inspection of the source code. Each weakness is categorized, assigned a severity rating, and traced to the specific file in which it was found.

4.1 Summary Table

#	Weakness	Category	Severity	Source
W1	No input sanitization on post content — stored XSS	Security	**High**	blog/views.py
W2	AI layer fails silently with no UI status indication	UX / Error Handling	**Medium**	blog/views.py
W3	Missing manage.py and requirements.txt in repository	Project Setup	**High**	blog/(missing)
W4	SQLite used as the database — no concurrency or encryption	Architecture	**High**	blog/settings.py
W5	Comment deletion by post authors has no audit trail	Security / UX	**Medium**	blog/views.py
W6	Hardcoded fallback SECRET_KEY and permanently enabled DEBUG	Security	**High**	blog/settings.py
W7	No pagination on notifications; bookmarks hard-limited to 10	Scalability	**Medium**	blog/views.py
W8	No file type, size, or integrity validation on image uploads	Security	**High**	blog/views.py
W9	All 27 view functions in a single monolithic views.py	Code Quality	**Medium**	blog/views.py
W10	No test suite anywhere in the project	Quality Assurance	**High**	blog/(absent)
W11	“No Comments Yet" Placeholder Not Removed on First AJAX Comment	UI/UX	**Low**	static/blog/js/post_interactions.js

4.2 Detailed Findings

W1 — No Input Sanitization on Post Content (Stored XSS)

Category: Security **Severity:** High

Post content is passed directly through markdown.markdown() and wrapped in mark_safe() with no HTML stripping. A logged-in user can embed malicious JavaScript in a post body, which then executes in every other visitor's browser — a classic stored XSS vulnerability.

```
rendered_body = mark_safe(markdown.markdown(post.content,
extensions=['fenced_code', 'tables']))
```

W2 — AI Layer Fails Silently with No UI Status Indication

Category: UX / Error Handling **Severity:** Medium

The OpenAI client initializes conditionally. When the key is absent, all AI functions return empty values or raise a ValueError caught silently by the calling view. No banner, flag, or message informs users or admins which features are functional.

```
client = OpenAI(api_key=settings.OPENAI_API_KEY) if settings.OPENAI_API_KEY else
None
```

W3 — Missing manage.py and requirements.txt in the Repository

Category: Project Setup **Severity:** High

Both manage.py (Django's CLI entry point) and requirements.txt (dependency manifest) were absent from the downloaded repository. Both had to be manually reconstructed, making the project non-reproducible from a fresh clone.

W4 — SQLite Used as the Database

Category: Architecture **Severity:** High

SQLite is a single-file, single-writer database with no concurrent write support, no access control, and no encryption at rest. The db.sqlite3 file is stored in the project root and is trivially copyable, exposing all user data including password hashes.

```
DATABASES = {'default': {'ENGINE': 'django.db.backends.sqlite3', 'NAME': BASE_DIR /
'db.sqlite3'}}
```

W5 — Comment Deletion by Post Authors Has No Audit Trail

Category: Security / UX **Severity:** Medium

Post authors can delete any comment on their posts, but no notification is sent to the original commenter. Although deleted_by and deleted_at fields are recorded via soft_delete(), the commenter has no visibility into the removal.

```
if (comment.author != request.user
    and comment.post.author != request.user
    and not request.user.is_superuser):
    return JsonResponse({'success': False, 'error': 'Not allowed'}, status=403)
```

W6 — Hardcoded Fallback SECRET_KEY and Permanently Enabled DEBUG

Category: Security **Severity:** High

If .env is absent, the application starts with a known insecure SECRET_KEY and DEBUG=True hardcoded. Running in debug mode on a public server exposes full stack traces, internal paths, environment variables, and query logs to any visitor who triggers an error.

```
SECRET_KEY = os.getenv('SECRET_KEY',
'django-insecure-change-this-in-production-xyz123')
```

```
DEBUG = True
```

W7 — No Pagination on Notifications; Bookmarks Hard-Limited to 10

Category: Scalability **Severity:** Medium

The notifications_list view loads all notifications in a single query with no upper bound. Separately, bookmarks in profile_detail are silently truncated to 10 entries with no indication to the user that more exist.

```
notifications =
Notification.objects.filter(user=request.user).select_related('actor',
'post').order_by('-created_at')
# bookmarks hard-limited elsewhere: .order_by('-created_at')[:10]
```

W8 — No Validation on Image Uploads

Category: Security **Severity:** High

Uploaded files are accepted without checking MIME type, file extension, file size, or whether the file is a valid image. This can exhaust disk space, cause application errors, or introduce file-processing vulnerabilities.

```
cover_image = request.FILES.get('cover_image')
gallery_files = request.FILES.getlist('gallery_images')
```

W9 — Monolithic views.py (27 Functions, 600+ Lines)

Category: Code Quality **Severity:** Medium

All 27 view functions covering at least seven distinct functional domains reside in a single file. This violates the Single Responsibility Principle, making the codebase difficult to navigate, test, or modify without unintended side effects.

W10 — No Test Suite

Category: Quality Assurance **Severity:** High

There is no tests.py file and no tests/ directory anywhere in the project. Every code change risks silently breaking existing functionality, with no automated safety net.

W11 — "No Comments Yet" Placeholder Not Removed on First AJAX Comment

Category: UI/UX **Severity:** Low

When a post has no comments, Django renders a static "No comments yet." placeholder. When the first comment is submitted via AJAX, the JavaScript appends the new comment to the list without removing the placeholder, causing both to appear simultaneously until the page is manually reloaded.

Step 5 — Re-Engineering Proposal

This section proposes ten concrete improvements that directly address the weaknesses in Step 4. Each proposal is specific, actionable, and includes implementation-ready code.

5.1 Summary Table

#	Proposal	Addresses	Effort	Category
P1	Sanitize post content with bleach to prevent XSS	W1	Low	Security
P2	Add AI availability flag and UI status banner	W2	Low	UX
P3	Commit manage.py, requirements.txt, and .env.example	W3	Trivial	Project Setup
P4	Migrate from SQLite to PostgreSQL	W4	Medium	Architecture
P5	Notify comment author when their comment is removed	W5	Low	Security/UX
P6	Enforce secure settings via startup env validation	W6	Low	Security
P7	Paginate notifications and remove the bookmark limit	W7	Low	Scalability
P8	Validate uploaded images for type, size, and integrity	W8	Medium	Security
P9	Refactor views.py into a feature-based views/ package	W9	Medium	Code Quality
P10	Write a core test suite covering permissions and CRUD	W10	High	QA
P11	Remove Placeholder Before Inserting First AJAX Comment	W11	Trivial	UI/UX

5.2 Detailed Proposals

P1 — Sanitize Post Content with bleach to Prevent XSS

Addresses: W1 **Effort:** Low

Introduce the bleach library to strip all unsafe HTML tags before Markdown-rendered content is passed to mark_safe(). Define a whitelist of permitted tags and apply bleach.clean() after markdown.markdown() in the post_detail view. Also add a max_length constraint to Post.content.

```
import bleach
```

```
ALLOWED_TAGS = bleach.sanitizer.ALLOWED_TAGS | {
    'p','h1','h2','h3','pre','code','table','thead','tbody','tr','th','td','img'
}
ALLOWED_ATTRIBUTES = {**bleach.sanitizer.ALLOWED_ATTRIBUTES, 'img': ['src','alt']}
rendered_body = mark_safe(
    bleach.clean(
        markdown.markdown(post.content, extensions=['fenced_code','tables']),
        tags=ALLOWED_TAGS,
        attributes=ALLOWED_ATTRIBUTES
    )
)
```

P2 — Add AI Feature Availability Flag and UI Status Banner

Addresses: W2 **Effort:** Low

Add a boolean flag in settings.py derived from the presence of the API key. Expose it globally via context_processors.py. Show a visible warning banner in the post creation form and a specific message in search results when AI is unavailable.

```
# settings.py
AI_ENABLED = bool(os.environ.get('OPENAI_API_KEY', ''))

# Template usage
{% if not ai_enabled %}
<div class="alert alert-warning">
  AI features are currently unavailable. Tags, SEO metadata,
  and semantic search are disabled.
</div>
{% endif %}
```

P3 — Commit manage.py, requirements.txt, and .env.example

Addresses: W3 **Effort:** Trivial

Commit the reconstructed manage.py and requirements.txt to the repository root. Add a .env.example file with placeholder values for all required environment variables, and update README.md with complete setup instructions.

```
# .env.example
SECRET_KEY=your-secret-key-here
DEBUG=True
OPENAI_API_KEY=your-openai-key-here
DEFAULT_FROM_EMAIL=noreply@example.com
DB_NAME=blogplatform
```

P4 — Migrate from SQLite to PostgreSQL

Addresses: W4 **Effort:** Medium

Replace the SQLite backend with PostgreSQL, which supports concurrent reads and writes, row-level locking, user-level access control, and encrypted connections. All existing migrations are already compatible — no model changes are required.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME', 'blogplatform'),
        'USER': os.environ.get('DB_USER', 'postgres'),
        'PASSWORD': os.environ.get('DB_PASSWORD', ''),
        'HOST': os.environ.get('DB_HOST', 'localhost'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
}
```

P5 — Notify Comment Author When Their Comment Is Removed

Addresses: W5 **Effort:** Low

Extend the delete_comment view to create a notification for the comment author whenever their comment is removed by someone other than themselves. Add a 'comment_removed' type to Notification.NOTIFICATION_TYPES. This reuses the existing create_notification() helper — no new infrastructure is needed.

```
# In delete_comment view, after soft_delete():
if comment.author != request.user:
    create_notification(
        user=comment.author,
        actor=request.user,
        post=comment.post,
        notif_type='comment_removed',
        message=f"Your comment on '{comment.post.title}' was removed by the post
author."
    )
```

P6 — Enforce Secure Settings via Startup Environment Validation

Addresses: W6 **Effort:** Low

Remove the hardcoded fallback for SECRET_KEY and read it exclusively from the environment. If SECRET_KEY is unset, exit immediately with a clear error. This prevents the platform from starting in an insecure configuration.

```
import sys
SECRET_KEY = os.environ.get('SECRET_KEY')
DEBUG = os.environ.get('DEBUG', 'False') == 'True'

if not SECRET_KEY:
    sys.exit('ERROR: SECRET_KEY is not set. Copy .env.example to .env and fill in
the values.')
if not DEBUG and 'insecure' in SECRET_KEY:
    sys.exit('ERROR: Insecure SECRET_KEY detected in non-debug mode.')
```

P7 — Paginate Notifications and Remove the Bookmark Hard-Limit

Addresses: W7 **Effort:** Low

Apply Django's built-in Paginator to the notifications_list view (20 per page). Remove the [:10] hard-limit on bookmarks in profile_detail and replace it with paginator-based navigation (10 per page). Reuse pagination controls already present in post_list.html.

```
# notifications_list view
paginator = Paginator(notifications, 20)
page_obj = paginator.get_page(request.GET.get('page'))

# profile_detail view — replace [:10]
paginator = Paginator(bookmarks, 10)
bookmark_page = paginator.get_page(request.GET.get('bookmark_page'))
```

P8 — Validate Uploaded Images for Type, Size, and Integrity

Addresses: W8 **Effort:** Medium

Create a reusable validator in blog/validators.py that checks extension against a whitelist, rejects files larger than 5 MB, and uses Pillow's Image.open().verify() to confirm the file is a valid image before saving it to disk.

```
from PIL import Image
import os
```

```
from django.core.exceptions import ValidationError

MAX_SIZE_MB = 5
ALLOWED_EXTENSIONS = ['.jpg', '.jpeg', '.png', '.webp']

def validate_image_upload(file):
    ext = os.path.splitext(file.name)[1].lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise ValidationError(f'Unsupported file type: {ext}.')
    if file.size > MAX_SIZE_MB * 1024 * 1024:
        raise ValidationError(f'File too large. Maximum is {MAX_SIZE_MB}MB.')
    try:
        img = Image.open(file)
        img.verify()
    except Exception:
        raise ValidationError('Uploaded file is not a valid image.')
```

P9 — Refactor views.py into a Feature-Based views/ Package

Addresses: W9 **Effort:** Medium

Split the monolithic views.py into a views/ package with one module per functional domain. The __init__.py re-exports all view names so that urls.py requires no changes — this is a pure refactor with zero functional impact.

```
blog/views/
    __init__.py        # re-exports all views — urls.py unchanged
    auth.py            # signup, login, logout, email verification
    posts.py           # post_list, post_detail, post_create, post_update,
post_delete
    interactions.py    # toggle_like, add_comment, delete_comment, toggle_bookmark
    profile.py         # profile_detail, profile_edit, analytics_dashboard
    notifications.py   # notifications_list
    search.py          # semantic_search
    ai_endpoints.py    # ai_tags, ai_seo
```

P10 — Write a Core Test Suite

Addresses: W10 **Effort:** High

Create a tests/ package inside the blog app with separate test files per domain, prioritizing authentication flows, post ownership rules, social interaction constraints, and soft-delete behavior.

```
class PostPermissionTests(TestCase):
    def test_non_owner_cannot_edit_post(self):
        owner = User.objects.create_user('owner', password='pass')
        other = User.objects.create_user('other', password='pass')
        post = Post.objects.create(title='T', content='C', author=owner,
status='published')
        self.client.login(username='other', password='pass')
        self.client.post(f'/post/{post.slug}/edit/', {'title': 'Hacked'})
        post.refresh_from_db()
        self.assertNotEqual(post.title, 'Hacked')

    def test_draft_hidden_from_guest(self):
        author = User.objects.create_user('author', password='pass')
        post = Post.objects.create(title='D', content='C', author=author,
status='draft')
        response = self.client.get(f'/post/{post.slug}/')
        self.assertEqual(response.status_code, 404)
```

P11 — Remove Placeholder Before Inserting First AJAX Comment

Addresses: W11 **Effort:** Trivial

Before inserting the new comment into the DOM, query for any `<li>` without an `id` attribute (the placeholder is the only such element in the list) and remove it. Also add `?v=2` to the script tag in `base.html` to bust the browser cache and ensure the fix is loaded immediately.

```
const placeholder = list.querySelector('li:not([id])'); if (placeholder) { placeholder.remove(); }
list.insertAdjacentHTML('beforeend', data.html);
```

Repository Link For The Modified Blog Platform

Repository URL	*https://github.com/Huzaifa-Kashif7/Blog-Platform-Modified*